

COSC 3P93 – Project Step 3 – Wiki Race

1 GROUP MEMBERS

Name	Student #
Geoffrey Jensen	7148710
Nicholas Parise	7242530
Stephen Stefanidis	7140030

CONTENTS

1	Group Members	1
	Contents	1
2	Introduction	1
3	Database Loading	2
4	PageRank	3
4.1	PageRank Sequential	3
4.2	PageRank OpenMP	3
4.2.1	Parallel Redesign	3
4.2.2	Performance Analysis	5
4.3	PageRank MPI	8
4.3.1	Parallel Redesign	8
4.3.2	Performance Analysis	9
5	Search	11
5.1	BFS Sequential	11
5.2	BFS OpenMP	11
5.2.1	Performance Analysis	13
5.3	BFS OpenMPI	13
5.3.1	Performance Analysis	14
5.4	Bidirectional BFS Sequential	14
5.5	Bidirectional BFS OpenMP	14
5.5.1	Performance Analysis	17
6	Compilation and Running	17
6.1	Database Installation	17
6.2	Compilation	17
6.3	PageRank	18
6.4	Search	18

2 INTRODUCTION

Our topic is Graph Processing, specifically processing and traversing large graphs. We were inspired by the game Wikipedia Race (also referred to as Wikipedia speedruns), where you

attempt to traverse from one Wikipedia page to another using only internal links. There are over 18 million Wikipedia pages, with 700 million links. The complexity of the problem is very high because of the large search space and the inconsistent number of links going to and from a page, meaning some of the graph is very sparse with a few central pages that connect wide parts of the graph.

For this part of the project, we had 3 main focuses. The first was to optimize the time it took to load the graph from our SQLITE database, the second was to parallelize the a PageRank algorithm, and the third was to parallelize search (BFS and bi-directional BFS).

3 DATABASE LOADING

Whenever the program starts, the graph must be read in from the SQLITE database. There are over 18 million pages, with 700+ million links. This is a lot of data to read and store in memory, which takes a significant amount of time. We optimized this loading time by using a producer-consumer threading pattern.

Inside the SQLITE database, each row is in the form of: [sourceId, destinations] where sourceId is a number, and destinations is a space seperated string in the form: "*destinationId₁ destinationId₂ destinationId₃ ...*". Each row must be parsed and converted into a vector<long>. This action in itself isn't that intensive, however since 18 million rows exist and have an uneven amount of destinations here is where the bottleneck lies.

SQLITE is limited to one read thread but the speed is mostly limited to the speed that the data can be parsed and allocated in memory. This isn't an issue though, since each link row can be processed independently, and it is bad practice to hold up the database read thread anyway. So the optimal solution is one read thread producer, with up to N threads to consume and parse string.

The main algorithm for reading in data is very simple. It consists of one Pthread that acts as the producer, the producer reads each row and adds that data to the queue until there are no rows left. This producer will signal the consumers when the size of the queue has passed a threshold value. Then there are N OPENMP threads that act as consumers. Each consumer will wait until there are enough elements in the queue, it then pops the batch size, and then processes all of the elements. These processed elements are added to each threads local map this reduces cache line misses and stops the need to lock the central map. The reason there is a batch size is to reduce the amount of communication and reduce the amount of time the queue is locked. After all the rows are processed and all the threads joined, the program synchronizes and combines all the local maps into one final finished map.

Execution Time (in seconds) 8 core, 32GB system w/NVME SSD		
Number of processes	Time (in seconds)	Improvement
Sequential performance	~ 20 seconds	
Parallel: 1 worker	~ 15 seconds	25% improvement
Parallel: 2 worker	~ 11 seconds	45% improvement
Parallel: 4 worker	~ 9 seconds	55% improvement
Parallel: 8 worker	~ 12 seconds	40% improvement

Table 1. Execution Times of Database Read

The execution time table above shows the difference in sequential and parallel performance. At 4 workers the system is running optimally, and with more it runs into bottlenecks reducing performance. These are most likely due to the number of hardware threads the system had access, read speed of NVME drive, along with the memory bandwidth needed for such an operation.

4 PAGERANK

This algorithm was developed by Google to determine how important each web page was, based on the quality of their incoming links. In the context of Wikipedia, some pages like Country are connected to many other important pages, while other pages have very little important connections. If a page is important, it means other important pages have links to it.

4.1 PageRank Sequential

The algorithm works like the following: A given web surfer can either follow the links on the current page, or randomly visit another page. This probability is called the Damping factor, and the value of $d=0.85$ is widely used. This means that there is an 85% chance the links on the current page will be used, and a 15% chance of randomly visiting another page. All page probabilities are initialized to $1 / |V|$, where $|V|$ is the total number of pages. Now PageRank will iteratively update these probabilities until convergence is reached. The next iteration of PageRanks is calculated by:

ALGORITHM 1: PageRank Sequential

Require: Graph $G = (V, E)$, threshold value t , Damping factor $d = 0.85$
 Initialize two arrays of length $|V|$, *prevRanks* and *nextRanks*
 2: Set all values in *prevRanks* to $1 / |V|$
 $diff = 1$
 4: **while** $diff > t$ **do**
 Set all values in *nextRanks* to $(1 - d) / |V|$
 6: **for** each node u of V **do**
 for each neighbor v of u **do**
 8: let $|u|$ be the number of neighbors of u
 $nextRanks[v] += d * prevRanks[u] / |u|$
 10: **end for**
 12: **end for**
 $diff = 0$
 for i from 0 to $|V| - 1$ **do**
 14: $diff = |prevRanks[i] - nextRanks[i]|$
 end for
 16: $prevRanks \leftarrow nextRanks$
end while

4.2 PageRank OpenMP

4.2.1 Parallel Redesign. Each iteration of PageRank takes the probability values from the *prevRanks* array and disperses them equally among that node's neighbours, into the *nextRanks* array. With how the sequential algorithm was created, the for loop on line 6

(Algorithm 1) can not be directly parallelized with OpenMP without the need of extra synchronization. This is because multiple iterations of the for loop could be trying to disperse their probabilities to the same index in *nextRanks* at the same time, creating a race condition. We *could* use a lock for every index of *nextRanks*, but this would be equal to 18 million locks for the full problem size. We thought this amount of locks was too much, so we decided to redesign the algorithm so we did not need them.

The major redesign of the OpenMP PageRank implementation is to switch from using a traditional outward adjacency list, to using an inward adjacency list. Now, in the algorithm itself, instead of iterating over *prevRanks* and dispersing the probabilities, we instead iterate over *nextRanks* and *collect* probabilities using the incoming connections. Since *prevRanks* is not modified and we are only performing read operations on it, we no longer need any forms of synchronization for our algorithm. The only caveat with changing the graph representation is that we still need the number of outgoing connections for each node. This was easy before, since we could simply take the length of the adjacency list, but with the inward list, we need to count and store the amount of outgoing connections separately at the start of the algorithm implementation.

ALGORITHM 2: PageRank OpenMP

Require: Inward Graph $G = (V, E)$, threshold value t , Damping factor $d = 0.85$
Initialize two arrays of length $|V|$, *prevRanks* and *nextRanks*
2: Set all values in *prevRanks* to $1 / |V|$
 $diff = 1$
4: **while** $diff > t$ **do**
 for each node u of V **do**
6: $nextRanks[u] = (1 - d) / |V|$
 for each **incoming** neighbor v of u **do**
8: let $|v|$ be the number of **outgoing** neighbors of v
 $nextRanks[u] += d * prevRanks[v] / |v|$
10: **end for**
 end for
12: $diff = 0$
 for i from 0 to $|V| - 1$ **do**
14: $diff = |prevRanks[i] - nextRanks[i]|$
 end for
16: $prevRanks \leftarrow nextRanks$
end while

The OpenMP parallel implementation involved using **#pragma omp parallel for** for major loop operations. The first loop that was parallelized is from line 2 of Algorithm 2, where the initial values of *prevRanks* are set to $1 / |V|$.

The next parallelized for loop is arguably the most important, which is the main loop on line 5 (2). We decided that a dynamic schedule was best for this loop. The number of incoming connections are in the range of $[0, 1597701]$, with order of nodes being random. This large range of incoming connections makes the amount of work very inconsistent between iterations. The overhead of a dynamic loop schedule is worth it to make the work more equal among each thread, with respect to the total number of incoming connections they each have to process.

The third loop to be parallelized is on line 13 (2), where we have to calculate the absolute difference between each index of *prevRanks* and *nextRanks* to check for convergence. This loop is parallelized using a parallel for directive as well, but it requires a reduction clause for the diff variable (**#pragma omp parallel for reduction(+:diff)**).

In the original implementation of the sequential PageRank algorithm, a `std::unordered_map` was used to map the ID's of the pages to the indices of the PageRank arrays. Whenever the code needed to access the *prevRanks* or *nextRanks* arrays, the ID from the graph would be used with the map to retrieve the index. This was brought forward to the OpenMP implementation, and it was not until the MPI version was being tested, did the performance implications of this constant hashing become apparent. Even though we look at access times of hash maps being $O(1)$ asymptotically, hashing is actually very expensive. To optimize the running time, all of the ID's in the graph were translated to those of the indices of the pages in the PageRank arrays, instead of constantly referencing them with the `unordered_map` on the fly. This improved the running time by 3-9 times! Instead of taking over an hour to finish, the algorithm would take 10 minutes (producing the same output).

4.2.2 Performance Analysis. For the OpenMP implementation, Speedup, Efficiency, and Cost metrics were calculated before and after the hashing optimization was made. (Figures 2, 3, 4, 5 are for unoptimized, and Figures 6, 7, 8, 9 are for optimized.) Interestingly, the Speedup was more pronounced when the code was doing the extra hashing work during the PageRank iterations, which is why we wanted to show both sets of metrics.

The runs for PageRank were done on a system with 6 cores and 16GB of RAM. Runs using 8 and 16 threads were included, but since the system only has 6 cores, the results are poor because of resource starvation.

Three problem sizes were tested, original, half, and quarter size. The data takes up too much space to run with double the problem size, so we ran it on reduced sizes instead. We define half the problem size to be a graph with half as many vertices, and half as many connections.

For the unoptimized version, there is a good speedup between using 1 and 2 threads. 2 threads had more than 90% efficiency. With 4 threads however we lose efficiency, which continues to decline with 8 and 16 threads. We had expected the efficiency of 4 threads to remain steady, but it was surprisingly not the case.

The size of the problem also did not have as big of an effect on the metrics as we expected. There is some sequential work that is done at the start of the algorithm (like creating an `unordered_map` of page ID's $\rightarrow$ indices). This sequential work also scales off of the problem size, so it makes sense for the time needed for the sequential portion of the code to scale with the problem size.

The metrics for the optimized version of the code tell a completely different story. First of all, the execution times are much shorter, which leaves less room for improvement by threading. When there was more computational work being done with the constant hashing, the threads made more of a difference. The speedup and efficiency numbers are not that impressive for the optimized version of the code, but a very slight increase in efficiency can be observed as the problem size increases, but not a large amount.

Execution Times					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1306	714	448	346	303
Half	2534	1385	889	665	576
Original	4591	2497	1608	1194	1033

Table 2. Execution Times of Unoptimized PageRank OpenMP

Speedup					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	1.82	2.92	3.77	4.31
Half	1.00	1.83	2.85	3.81	4.40
Original	1.00	1.84	2.86	3.85	4.44

Table 3. Speedup of Unoptimized PageRank OpenMP

Efficiency					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	0.91	0.73	0.47	0.27
Half	1.00	0.91	0.71	0.48	0.27
Original	1.00	0.92	0.71	0.48	0.28

Table 4. Efficiency of Unoptimized PageRank OpenMP

Cost					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1306	1428	1792	2768	4848
Half	2534	2770	3556	5320	9216
Original	4591	4994	6432	9552	16528

Table 5. Cost of Unoptimized PageRank OpenMP

Execution Times					
	Number of Threads				
PB	1	2	4	8	16
Quarter	137	107	94	86	83
Half	274	212	183	164	154
Original	582	436	367	323	298

Table 6. Execution Times of Optimized PageRank OpenMP

Speedup					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	1.28	1.46	1.59	1.65
Half	1.00	1.29	1.50	1.67	1.78
Original	1.00	1.33	1.59	1.80	1.95

Table 7. Speedup of Optimized PageRank OpenMP

Efficiency					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	0.64	0.36	0.20	0.10
Half	1.00	0.65	0.37	0.21	0.11
Original	1.00	0.67	0.40	0.23	0.12

Table 8. Efficiency of Optimized PageRank OpenMP

Cost					
	Number of Threads				
PB	1	2	4	8	16
Quarter	137	214	376	688	1328
Half	274	424	732	1312	2464
Original	582	872	1468	2584	4768

Table 9. Cost of Optimized PageRank OpenMP

4.3 PageRank MPI

4.3.1 Parallel Redesign. There is a lot of data dependencies in the PageRank algorithm. The major offender being that a complete copy of the *prevRanks* array is required to calculate *nextRanks*. If we wanted to split the PageRank arrays across different processes, then a lot of communication would be required, incurring performance loss. Thankfully, we can have each process store their own copy of *prevRanks* and *nextRanks*, while they only own a partition of the graph.

The design of our distributed parallel PageRank works off the assumption every node has the complete copy of the *prevRanks* array. With a complete copy, each node can use their partition of the graph to calculate their partial *nextRanks* array. Then, `MPI_Allreduce` can be used to synchronize the *nextRanks* array across all processes, which later becomes the *prevRanks* array in the next iteration.

ALGORITHM 3: PageRank MPI

Require: Graph $G = (V, E)$, threshold value t , Damping factor $d = 0.85$
 Partition graph across all processes using `MPI_Send()` and `MPI_Recv()`
 2: Initialize two arrays of length $|V|$, *prevRanks* and *nextRanks* on every process
 Set all values in *prevRanks* to $1 / |V|$
 4: $diff = 1$
while $diff > t$ **do**
 6: MASTER: Set all values in *nextRanks* to $(1 - d) / |V|$
 WORKER: Set all values in *nextRanks* to 0
 8: **for** each node u of local V **do**
 for each neighbor v of u **do**
 10: let $|u|$ be the number of neighbors of u
 nextRanks [v] $+= d * prevRanks[u] / |u|$
 12: **end for**
 14: **end for**
 14: `MPI_Allreduce(nextRanks)`
 $diff = 0$
 16: **for** i from 0 to $|V| - 1$ **do**
 $diff = |prevRanks[i] - nextRanks[i]|$
 18: **end for**
 `MPI_Allreduce(diff)`
 20: *prevRanks* $\leftarrow nextRanks$
end while

The first line of the PageRank MPI algorithm (3) partitions the graph equally among the processes. In the implementation, the communication is done using only `MPI_Send()` and `MPI_Recv()`. The master process will communicate to each process the total number of pages in the graph, and how many pages that process will be responsible for. Then, the master will send over the graph information that has already been mapped to the PageRank indices. All the workers have to do in each iteration is to look at the indices in their local graph, and compute their local *nextRanks* array. To calculate the *diff* value used in the convergence check, each process calculates their local *diff* for their partitioned pages, then another `MPI_Allreduce` is used to synchronize the global *diff* value.

4.3.2 *Performance Analysis.* Figures 11, 12, 13, 14 show the metrics for the PageRank MPI algorithm. Despite adding more processes, the execution times got worse for smaller problem sizes, and only slightly better for the original size.

To see why adding more processes was barely improving the execution time, we created visualizations of the time the program was doing computational work vs when it was communicating (shown in Table 10).

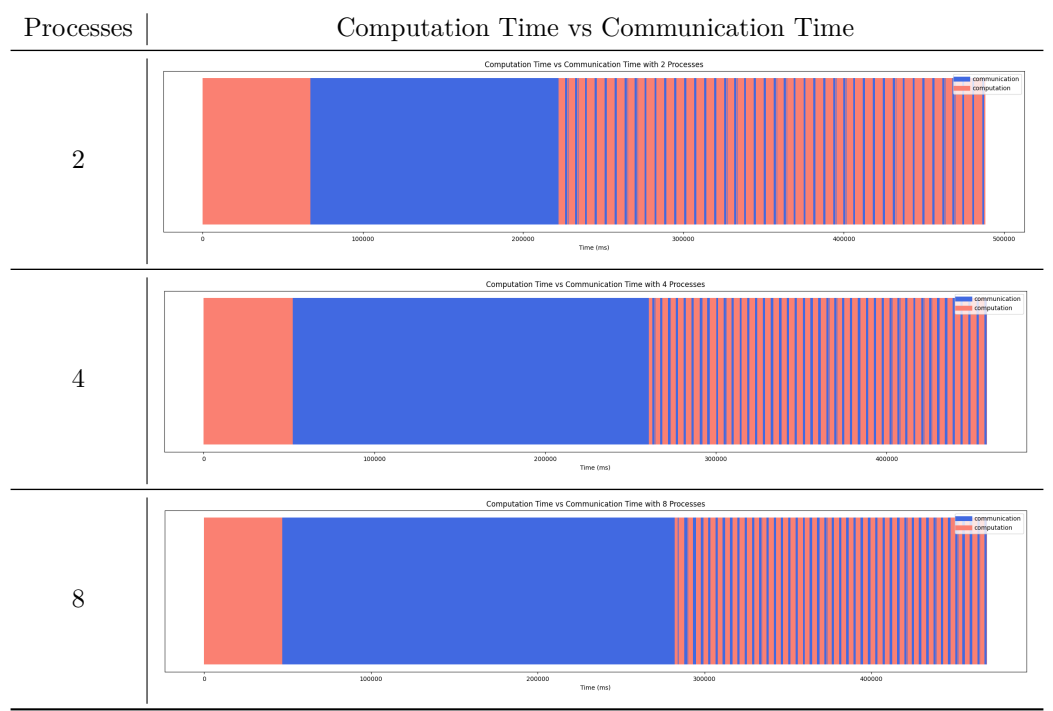


Table 10. Computation Time vs Communication Time over the Lifespan of the Program

The first red chunk of computation time is when the master process is mapping the graph ID's to indices, and it's creating it's own local graph partition. The next big blue chunk is when the master is communicating the graph partitions to each process. This is by far the most expensive operation, but it is necessary to communicate the graph partitions. After the massive communications chunk, the actual PageRank iterations begin. Here there is repeating pattern of a large computation chunk, which is when the local *nextRanks* are being calculated, then a thick blue line, which is the MPI_Allreduce of the *nextRanks* array, then there is a sliver of computation time which is calculating the local *diff* values, followed by a very thin communications slice, which is doing MPI_Allreduce on the *diff* value.

These visualizations show that as the number of processes increase, less time is needed for computation, but more time is needed for communications. This tradeoff is why the execution times barely improved when more processes were added.

Execution Times				
	Number of Processes			
PB	1	2	4	8
Quarter	180	193	197	203
Half	279	300	302	312
Original	523	489	459	469

Table 11. Execution Times of PageRank MPI

Speedup				
	Number of Processes			
PB	1	2	4	8
Quarter	1.00	0.93	0.91	0.88
Half	1.00	0.93	0.92	0.89
Original	1.00	1.07	1.14	1.12

Table 12. Speedup of PageRank MPI

Efficiency				
	Number of Processes			
PB	1	2	4	8
Quarter	1.00	0.47	0.23	0.11
Half	1.00	0.47	0.23	0.11
Original	1.00	0.53	0.28	0.14

Table 13. Efficiency of PageRank MPI

Cost				
	Number of Processes			
PB	1	2	4	8
Quarter	180	386	788	1624
Half	279	600	1208	2496
Original	523	978	1836	3752

Table 14. Cost of PageRank MPI

5 SEARCH

This section will discuss the methods that were used to find a path from article A to B as quickly as possible. In total, we developed five search algorithms over vanilla BFS and Bidirectional BFS. We implemented a sequential and OpenMP version for each and an OpenMPI approach for vanilla BFS.

5.1 BFS Sequential

While sequential BFS was described in detail in the second part of the assignment, this section acts as a quick refresher and a way to find opportunities for parallelism. The sequential approach that we used for part 2 is described in Algorithm 4. The largest opportunity for parallelization is the while loop on line 4 which pops the next node from the queue and adds all of its immediate connections to the queue. However, this loop will need to be slightly modified to allow the searcher to asynchronously pop nodes while ensuring that the parallel BFS still maintains the property of finding optimal paths by exploring depth-by-depth. We will explain these modifications in the next section.

ALGORITHM 4: Breadth-First Search (BFS)

Require: Graph $G = V, E$, starting node s , end node e

Ensure: Order of shortest path from s to e

```

1: Initialize a queue  $Q$ 
2: Initialize a parent map  $P$ 
3: Mark all nodes as unvisited
4: Enqueue  $s$  into  $Q$  and mark  $s$  as visited
5: while  $Q$  is not empty do
6:    $u \leftarrow$  Dequeue  $Q$ 
7:   if  $u$  is  $e$  then
8:     Exit loop
9:   end if
10:  for each neighbor  $v$  of  $u$  do
11:    if  $v$  is unvisited then
12:      Mark  $v$  as visited
13:       $P[v] \leftarrow u$ 
14:      Enqueue  $v$  into  $Q$ 
15:    end if
16:  end for
17: end while
18: Use  $P$  to trace the final path

```

5.2 BFS OpenMP

The reason the sequential BFS exhibits the property of going depth-by-depth is that it uses a queue to ensure that close neighbours to the starting node end up visited first. However, we are unable to rely on solely a queue for the parallel approach because we have no way of knowing in which order nodes will be added or popped from the queue. To maintain correctness by keeping this algorithm close to the sequential version, we used two vectors called *frontier* and *next_frontier* that contain an unordered array of nodes in the current

depth and nodes in the next depth respectively. We will need to add a for loop inside the while loop on line 4 that visits each of the nodes inside our current depth. Finally, we will need to be aware of all critical regions that may lead to race conditions. This includes the block of code that updates the parent of the current node, and adds the node to the next frontier. Our final parallel BFS algorithm is Algorithm 5. In line 7 we used a parallel for

ALGORITHM 5: Parallel BFS (Shared Memory)

Require: Graph $G = V, E$, starting node s , end node e
Ensure: Order of shortest path from s to e

```

1: Initialize a vector  $F$ 
2: Initialize a parent map  $P$ 
3: Mark all nodes as unvisited
4: Enqueue  $s$  into  $F$  and mark  $s$  as visited
5: while  $F$  is not empty do
6:   Initialize next frontier vector  $NF$  with nothing
7:   #pragma omp parallel for schedule(dynamic)
8:   for  $i$  from 0 to  $F.size$  do
9:      $u \leftarrow F[i]$ 
10:    if  $u$  is  $e$  then
11:      Exit loop
12:    end if
13:    for each neighbor  $v$  of  $u$  do
14:      if  $v$  is unvisited then
15:        #pragma omp critical
16:        {
17:          Mark  $v$  as visited
18:          Add  $v$  to  $NF$ 
19:        }
20:      end if
21:    end for
22:  end for
23:   $F \leftarrow NF$  {This moves the search to the next depth level}
24: end while

```

loop because each node expansion is mostly independent, the only thing that needs to be synchronized is the process of marking nodes as visited and adding them to the next frontier. For this loop we used a dynamic scheduling because the graph is not load balanced at all, for example if you look at each node as a Wikipedia page, one page might have only 10 outgoing links, while another page has thousands. Because each frontier is so difficult to predict, we used a dynamic scheduler to make the frontier expansion more adaptive. This decision led to a slight increase in performance, for instance using a default scheduling determined there is no path from Mario to Beatrice Ombuki-Berman in 18 seconds, while it took only 13 seconds using a dynamic scheduling. The reason we switched from using queues to vectors is that doing the loop on line 8 is not possible with queues because you can only access the end of a queue. A critical section was used on line 15 because adding to vectors is not thread safe because vectors can grow in size, and we have no way to efficiently know how many nodes are in the next frontier.

Execution Times (in seconds)					
	Number of Threads				
PB	1	2	4	8	16
Quarter	12.4	6.8	6	3.8	4.8
Half	21.4	10	6	7.2	6
Original	26.8	22.6	9.4	10	11.2

Table 15. Execution Times of BFS OpenMP

Speedup					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	1.82	2.07	3.26	2.58
Half	1.00	2.14	3.57	2.97	3.57
Original	1.00	1.19	2.85	2.68	2.39

Table 16. Speedup of BFS OpenMP

Efficiency					
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.0	0.91	0.52	0.41	0.16
Half	1.0	1.0	0.89	0.37	0.22
Original	1.0	0.60	0.71	0.34	0.15

Table 17. Efficiency of BFS OpenMP

5.2.1 Performance Analysis. This analysis was conducted by randomly picking two articles from each problem size, recording the times and repeating that 5 times, taking the average times from these experiments. One thing to note is that using 2 threads for the half size problem size led to a speedup of 2.14, which is not feasible. This outlier was probably caused by this run picking luckier articles which had an easier path to find. The smaller problem sizes were created by sampling articles from the full size, and using the subset as the graph. This process leads to a lot of connections being useless and makes the smaller sizes a lot less connected than the actual wiki graph. Due to this, we would have to run the random article selector until we found five pairs that actually had a path between them.

5.3 BFS OpenMPI

The distributed approach to BFS was a little bit more complicated than the shared memory approach due to explicit message passing being required, and needing to think about how to divide the work across processes. While OpenMP tries to explore all of the nodes in parallel, the MPI approach is to distribute the graph across a team of processes by dividing the vertex set into a roughly equal amount of subsets. Each process stores the following local collections: frontier, next frontier, parent map, and adjacency list. The next frontier

Cost					
	Number of Threads				
PB	1	2	4	8	16
Quarter	12.4	13.6	24	30.4	76.8
Half	21.4	20	24	57.6	96
Original	26.8	45.2	37.6	80	179.2

Table 18. Cost of BFS OpenMP

and parent map are then consolidated in an all-to-all communication at the end of every iteration. Aside from some implementation details, that’s pretty much the only difference from sequential and distributed BFS. Of course, the devil is in the details, because ensuring proper communication between the processes proved to be a challenge. One problem with the approach is that some work is sure to be repeated because one rank can’t possibly know the visited status of an unowned node during the expansion. The only theoretical way to avoid this issue would be communicating with the owner rank for every node expanded (which is clearly infeasible due to latency). This issue leads to the next concern which is the occurrence of out-of-memory errors. Because a node can appear in multiple next frontiers, which have exponential growth, my laptop with 16GB of RAM would frequently run out of memory for two vertices that are more than three depths apart (which is a problem that we could not fix in time). Algorithm 6 is the final implementation that was used for the distributed approach.

5.3.1 Performance Analysis. For testing the performance of MPI BFS, we did not use different problem sizes as we found the method used in the previous section to be a little misleading, and we did not want to deal with out-of-memory errors for large random paths. For comparing performance of the distributed BFS we used a fixed source and destination of the moon to parallel computing with just one run each. We made two tables, one considering the time it takes to partition the nodes across the ranks (Table 19), and one that purely measures the BFS computation (Table 20). The computer used for these tasks could only handle up to 4 processes.

5.4 Bidirectional BFS Sequential

The sequential version of Bidirectional BFS (BiBFS) was discussed but not implemented in the previous project stage. So in this stage, both the sequential and parallel versions of BiBFS were implemented. BiBFS is essentially an extension of regular BFS but with two searchers - one that starts searching from the beginning page and one that starts from the goal page. The idea is that eventually they will find a common page in the middle, at which point the search is complete. I won’t discuss the sequential version to great detail because it is pretty much just two copies of regular BFS. One concern with our project is that finding the incoming edges is necessary for the backwards searcher, and this process is very expensive for the Wikipedia graph. The implementation uses Algorithm 7.

5.5 Bidirectional BFS OpenMP

The shared memory implementation of BiBFS attempts to run both sides of the search in parallel. It uses the same procedure as Algorithm 7 but adds a `#pragma omp parallel` section around the forward and backward expansion. This allows the two to be run in parallel to

ALGORITHM 6: Parallel BFS (Distributed Memory)

Require: Graph $G = (V, E)$, starting node s , end node e

- 1: Partition graph across all processes using `MPI_Send()` and `MPI_Recv()` (each rank r owns V_r)
- 2: Initialize two vectors of length $|V_r|$, *frontier* and *next_frontier* on every process
- 3: Initialize a parent map on every process
- 4: Add s to the frontier of r who owns s
- 5: **while** e not found **and** depth limit not exceeded **do**
- 6: Initialize an array of *size* buckets to store next frontier
- 7: Initialize an array of *size* parent buckets to store parent of nodes in next frontier
- 8: {Each r expands their local frontier F_r }
- 9: **for** each node u of F_r **do**
- 10: **for** each neighbor v of u **do**
- 11: **if** v does not have a parent **then**
- 12: Add v to bucket r where $v \in V_r$
- 13: Add u to parent bucket r where $u \in V_r$
- 14: **end if**
- 15: **end for**
- 16: **end for**
- 17: {only one comm. is required for both buckets because they are the same length}
- 18: `MPI_Alltoall()` to share the size of each bucket
- 19: Two `MPI_Alltoallv()`'s to share the bucket and parent buckets with each r
- 20: **for** each node-parent pair v, u of received buckets **do**
- 21: **if** v does not already have a parent **then**
- 22: Add v to local next frontier NF_r
- 23: $parent[v] \leftarrow u$
- 24: **end if**
- 25: **end for**
- 26: **if** e not found **then**
- 27: $F_r \leftarrow NF_r$
- 28: **end if**
- 29: **end while**

Number of Processes	Time (s)	Speedup	Efficiency	Cost
1	64.3296	1.000	1.000	64.3296
2	49.8259	1.291	0.646	99.6518
4	35.0117	1.837	0.459	140.0468

Table 19. Parallel BFS (Distributed Memory) - Total

some degree, however a critical section is required to update both queues. This need for a critical section reduces the search to run almost sequentially. meaning that parallel BiBFS performed the worst out of all the search algorithms. The only benefit of using this method is that sometimes if a path does not exist, BiBFS will figure this out earlier because it starts from the goal page as well. Because the parallel implementation performed so poorly, and

Number of Processes	Time (s)	Speedup	Efficiency	Cost
1	8.6493	1.000	1.000	8.6493
2	6.2032	1.396	0.697	12.4065
4	4.8924	1.770	0.443	19.5697

Table 20. Parallel BFS (Distributed Memory) - BFS Computation

ALGORITHM 7: Bidirectional Breadth-First Search (BiBFS)

Require: Graph $G = V, E$, starting node s , end node e
Ensure: Order of shortest path from s to e

- 1: Initialize a forward queue Q_f and backward queue Q_b
- 2: Initialize a forward parent map P_f and backward parent map P_b
- 3: Enqueue s into Q_f and mark s 's parent as -1 in P_f
- 4: Enqueue e into Q_b and mark e 's parent as -1 in P_b
- 5: **while** Q_f is not empty **and** Q_b is not empty **do**
- 6: **while** Q_f has nodes of current depth **do**
- 7: $u \leftarrow$ Dequeue Q_f
- 8: **for** each neighbor v of u **do**
- 9: **if** v has no parent **then**
- 10: $P_f[v] \leftarrow u$
- 11: Enqueue v into Q_f
- 12: **end if**
- 13: **if** $v \in P_b$ **then**
- 14: Record meeting node $mid \leftarrow v$
- 15: Exit loop
- 16: **end if**
- 17: **end for**
- 18: **end while**
- 19: **while** Q_b has nodes of current depth **do**
- 20: $u \leftarrow$ Dequeue Q_b
- 21: **for** each neighbor v of u **do**
- 22: **if** v has no parent **then**
- 23: $P_b[v] \leftarrow u$
- 24: Enqueue v into Q_b
- 25: **end if**
- 26: **if** $v \in P_f$ **then**
- 27: Record meeting node $mid \leftarrow v$
- 28: Exit loop
- 29: **end if**
- 30: **end for**
- 31: **end while**
- 32: **end while**
- 33: Use P_b to trace the path from e to mid and P_f to trace the path from mid to s

the structure does not work well with distributed memory, OpenMPI was not attempted for BiBFS.

5.5.1 Performance Analysis. Similarly to the last section, only the full graph was considered for the performance tests. We also used the same path of moon to parallel computing. It’s also worth noting that these times do not include the time for loading the incoming edges (which took significantly longer than outgoing links). As you can see from Table 21, the parallel BiBFS offers no improvement over sequential due the reasons discussed above.

Number of Threads	Time (s)	Speedup	Efficiency	Cost
1	58.8683	1.000	1.000	58.8683
2	55.0430	1.069	0.535	110.0860
4	50.9232	1.156	0.289	203.6928
8	61.2653	0.961	0.120	490.1224
16	58.9993	0.998	0.062	943.9888

Table 21. Parallel BiBFS (Shared Memory)

6 COMPILATION AND RUNNING

Github Repository: <https://github.com/Nicholas-Parise/WikiRace> (includes instructions on importing the database yourself if you need to – though it is not recommended as it takes a long time)

6.1 Database Installation

If you still have the same SQLite file from stage 2 you can reuse that, if not follow these instructions:

- Download SQLite database from Google Drive:
– <https://drive.google.com/file/d/1ictDdg47LH-8rf1LApjV6BU3xMDISp0z/view?usp=sharing>
- Place the .gz file into the project root directory
- Decompress .gz file using gunzip, 7Zip, or WinRAR

6.2 Compilation

The program’s can be compiled using CMAKE. All dependencies are included, but OPEN MPI must be installed on the host machine (if it is not already). The executables can be compiled on Windows and Linux. Starting from the root project directory, running the following commands will compile the executables:

```
cd build
cmake ..
cmake --build .
```

Listing 1. Compiling Bash Commands

Once compiled, any of the executables in the build folder can be selected and ran. If something goes wrong, ensure that the uncompressed sqlite file ‘wikipedia.sqlite’ is in the

:18

project root folder (or exactly one folder before the executable) and that it contains roughly 8.7GB of data.

6.3 PageRank

The PageRank executables take in command-line arguments. Here are sample executions.

Shared Memory (OpenMP) with 4 threads, and full problem size. For half the problem size, replace 1 with 0.5.

```
./pageRankSharedMemory.exe 4 1
```

Listing 2. Shared Memory PageRank Sample Execution

Distributed Memory (OpenMPI) with 2 processes, and full problem size. Replace 1 with 0.5 for half the problem size.

```
mpirun -n 2 ./pageRankDistributedMemory.exe 1
```

Listing 3. Distributed Memory PageRank Sample Execution

6.4 Search

For any shared memory program the default option is to use 16 threads but you can change this by modifying the NUM_THREADS constant in the header file graph.h. It is also recommended that when it prompts you to "Enter number of pages", just leave your input blank and press enter as that was just for the performance analysis problem size table.

The sequential and OpenMP BFS can be tested with the 'wikiRace' executable. The program will prompt you to select sequential or parallel once you select two articles.

```
./wikiRace
```

Listing 4. Seq. or Shared Memory BFS Execution

The OpenMPI BFS can be tested with the 'mpi_wikiRace' executable. Please note that due to the issues discussed in section 5.3, the program may crash if the articles you choose are too far apart.

```
mpirun -n <number-of-processes> ./mpi_wikiRace
```

Listing 5. Distributed Memory BFS Execution

The sequential and OpenMP Bidirectional BFS can be tested with the 'bidirectional_wikiRace' executable. The program will prompt you to select sequential or parallel once you select two articles.

```
./bidirectional_wikiRace
```

Listing 6. Seq. or Shared Memory BiBFS Execution